IN4120 - Search Technology

SKIP LISTS

A PROBABILISTIC ALTERNATIVE TO BALANCED TREES

Based on the research paper by William Pugh

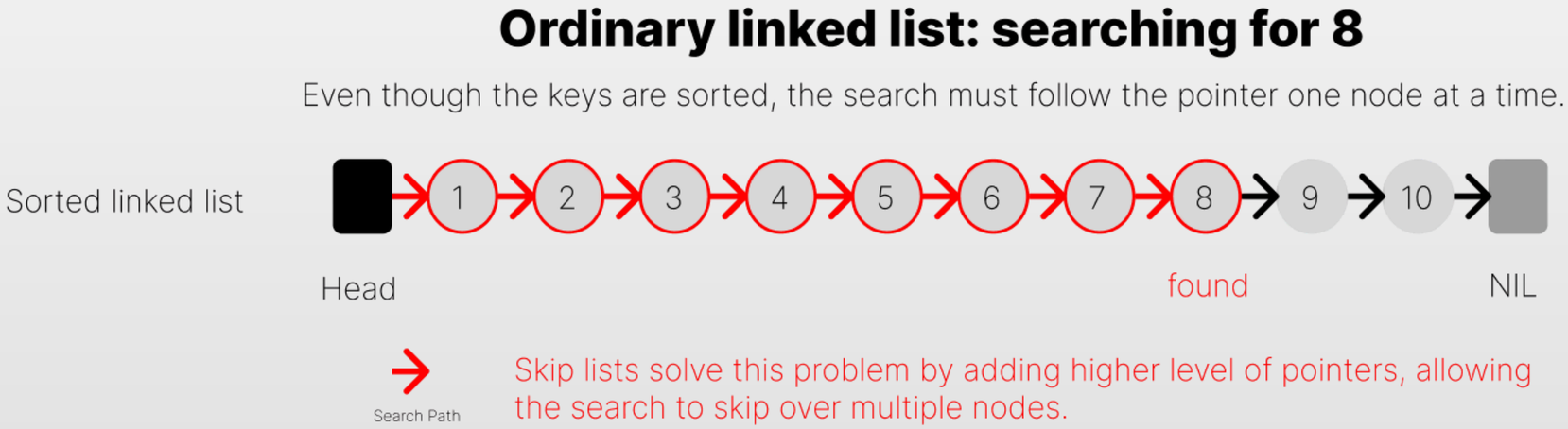
WHAT ARE SKIP LISTS ?

Introduction

- The skip list is a data structure designed to store and retrieve elements efficiently
- The skip list contains several layers of sorted linked lists → the upper layers help the search operation jump over many elements
- Unlike balanced search trees, skip lists use randomization instead of strict balancing rules.
- There are three primary operations in the skip list: search, insert, and delete
- Search, insertion, and deletion have an expected time complexity of O(log n).

THE PROBLEM WITH ORDINARY LINKED LISTS

- In an ordinary linked list, each node contains a value and a pointer to the next node
- Even when the list is sorted, searching requires checking elements sequentially from the beginning
- In the worst case, finding an element requires examining all nodes, resulting in a time complexity of O(n)
- Skip lists address this limitation by introducing additional levels of pointers, allowing searches to skip over multiple elements.



HOW SKIP LISTS WORK

Multiple levels

A skip list consists of several levels of sorted linked lists

Bottom level

Level 0 contains all elements in sorted order, just like an ordinary linked list

Higher levels

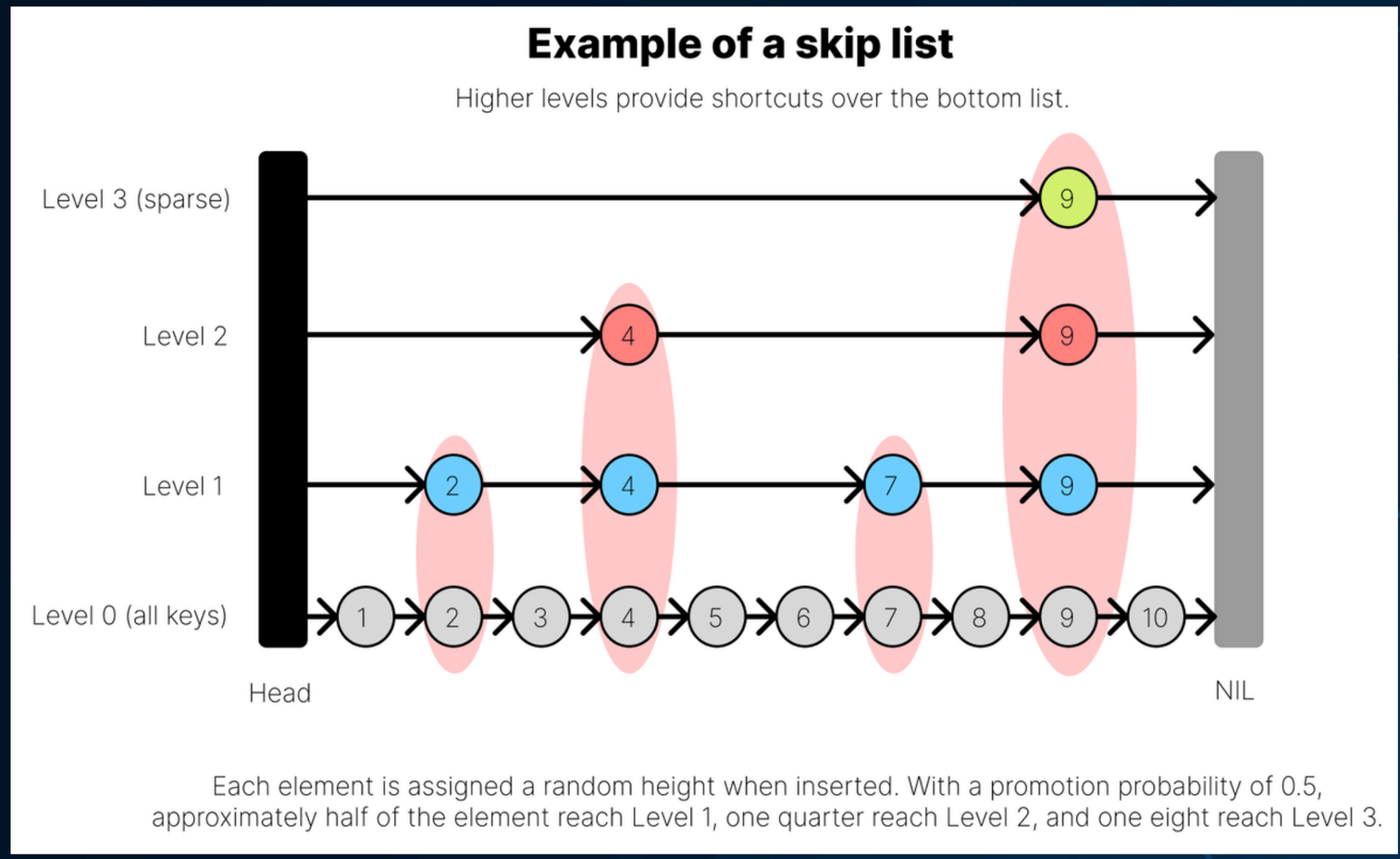
Each higher level contains a subset of the elements, allowing searches to skip over multiple nodes

Randomization

When a new element is inserted, it's height is determined randomly, typically using a probability of 0.5 to promote it to each additional level

Efficient searching

By using these shortcuts, skip lists achieve an expected seach time complexity of O(log n)



SEARCHING IN A SKIP LIST

Start at the top

Begin at the highest level, starting from the head node

Move right

Follow the pointers while the next element is smaller than the target.

Move down

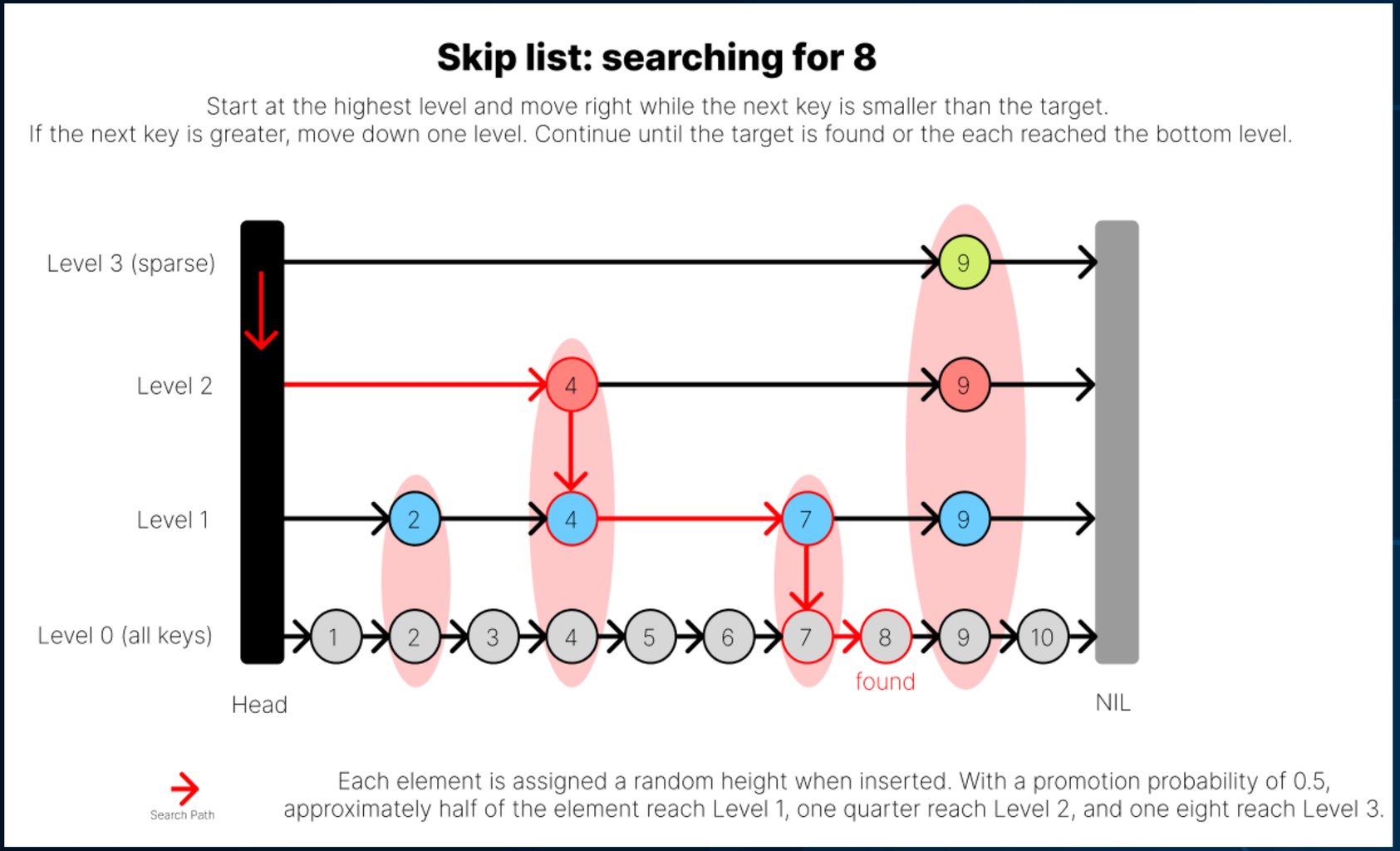
If the next element is greater than the target, move down one level

Repeat

Continue moving right and down until the target is found or the search reaches the bottom level

Efficiency

Skip lists achieve an expected search time complexity of O(log n), compared to O(n) for ordinary linked lists



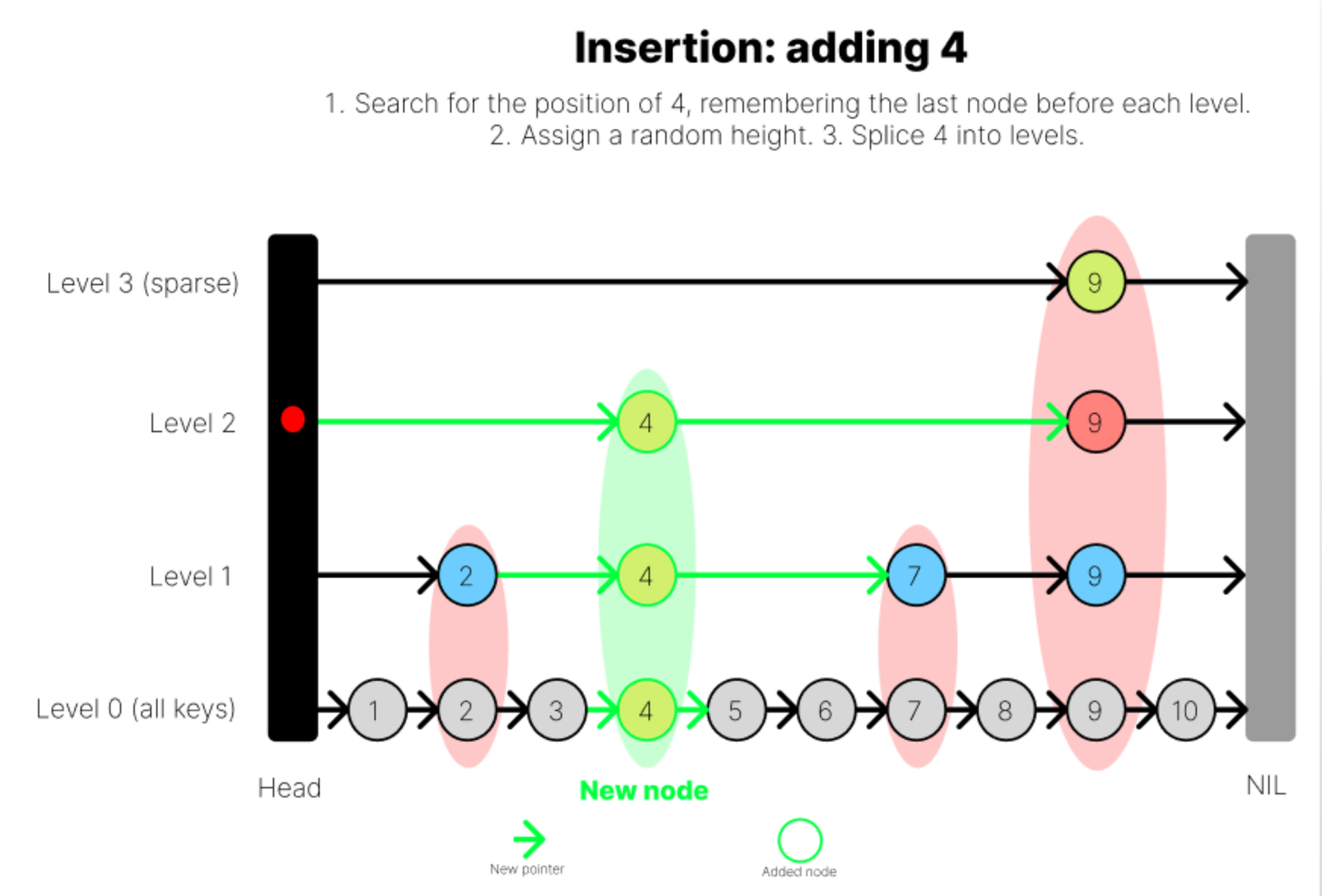
INSERTION IN SKIP LISTS

Insertion (Adding an element)

- Search for the correct position of the new element.
- Assign the new element a random height.
- Update the pointers at each relevant level to include the new element.

Performance

- Both insertion and deletion have an expected time complexity of O(log n).



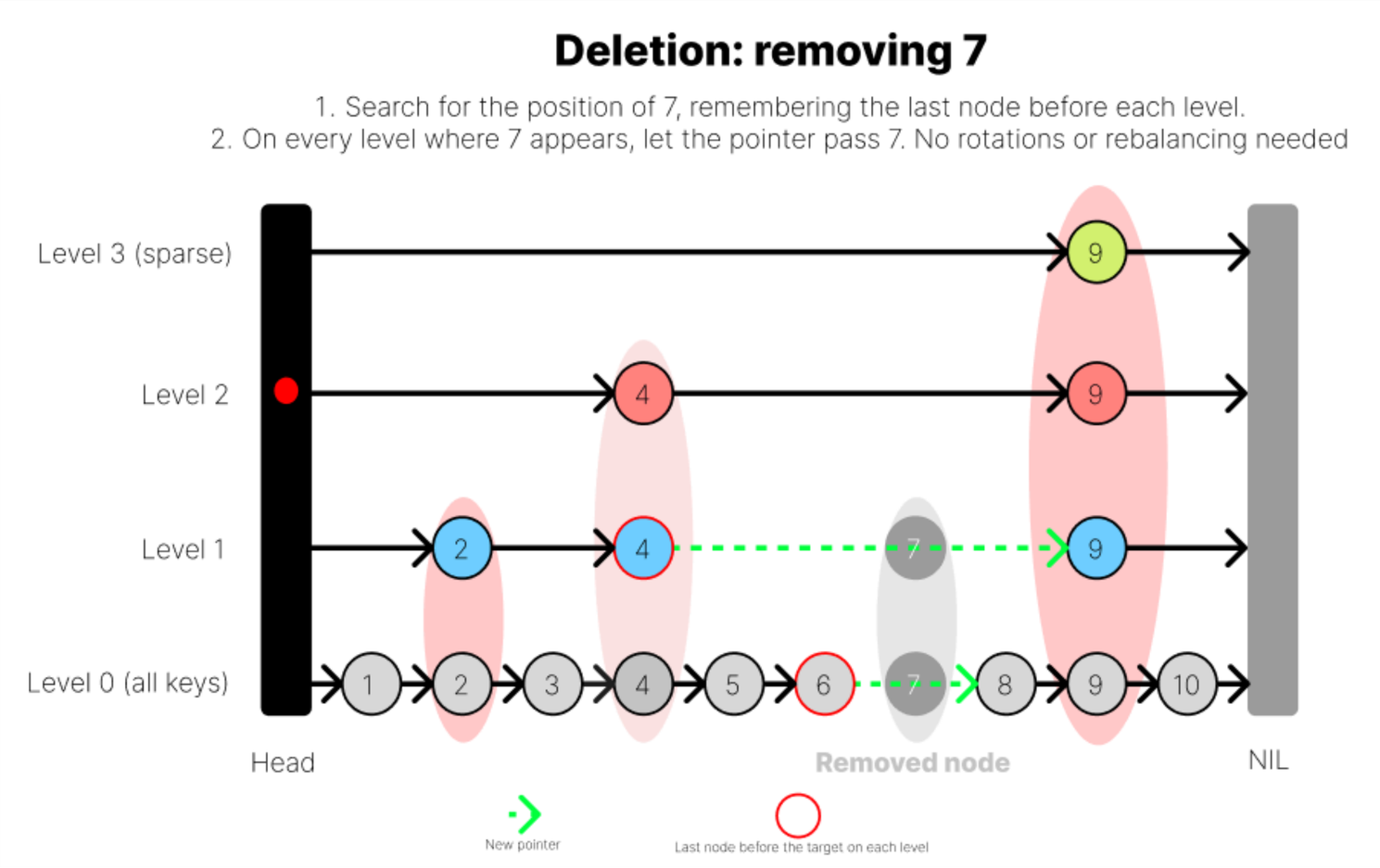
DELETION IN SKIP LISTS

Deletion (Removing an element)

- Search for the element that should be removed.
- Update the pointers at every level where the element appears.
- Remove the element without requiring tree rotations or rebalancing.

Performance

- Both insertion and deletion have an expected time complexity of O(log n).



PERFORMANCE AND TIME COMPLEXITY

Expected performance

Skip lists support search, insertion, and deletion in O(log n) expected time.

Worst-case performance

Operations may take O(n) time if the randomly generated structure is unfavorable.

Comparison with balanced trees

Skip lists offer similar asymptotic expected performance without requiring complex rotations or rebalancing.

Space complexity

Skip lists require O(n) expected space to store their nodes and additional forward pointers.

SKIP LISTS IN SEARCH TECHNOLOGY

1. Skip Pointers in Posting Lists

- Search engines make use of inverted indices that contain lists of document IDs for every term.
- Skip pointers enable the search algorithm to skip certain document IDs when intersecting posting lists (AND queries).
- Thus, there will be fewer comparisons required for processing specific queries.

2. Multi-Level Skipping in Lucene

- Apache Lucene makes use of multi-level skip information within the inverted index to navigate postings.
- It works by following the same principle as that of skip lists, which is skipping unnecessary traversing.

3. Practical Applications

- Redis: Makes use of skip lists to process operations on sorted sets.
- LevelDB/RocksDB: Use skip-list-based data structures in memory storage of their databases.

Key takeaway: *The skip lists concept is useful for search technologies as it allows skipping unnecessary items to increase efficiency.*

ADVANTAGES AND LIMITATIONS

Advantages

- **Simple Implementation:** Skip lists are easy to implement compared to several other types of balanced search trees.
- **Efficiency:** Search, insert, and delete operations take O(log n) expected time.
- **No need for rebalancing:** Height randomness avoids any need for tree rotation and balancing.

Limitations

- **No worst-case guarantee:** Operations can take O(n) time in unfavorable cases.
- **Use of additional memory:** More than one level requires additional forward pointers to be used.
- **Randomized performance:** The structure and performance depend on randomly assigned node heights.

CONCLUSION

- **Efficient searching:** Skip lists employ different layers of pointers which help in skipping elements and minimize the number of moves in a search process.
- **Randomized structure:** The height of nodes is assigned randomly, thus helping the skip list structure preserve its form without complicated balancing procedures.
- **Expected performance:** Operations such as search, insertion, and deletion have O(log n) expected time complexity, but the worst case is still O(n).
- **Alternative to balanced trees:** The skip list offers simpler implementation and comparable expected time complexity.

Key takeaway: Skip lists demonstrate how randomization can be used to achieve efficient searching and dynamic updates without complex balancing algorithms.

THANK YOU

References

Pugh, W. (1990). Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM, 33(6), 668–676*.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.